

CS202

SOFTWARE TOOLS AND TECHNIQUES FOR CSE

ASSIGNMENT - 1

PREPARED BY

Gaurav Budhwani

22110085

ASSIGNMENT - 1.....	1
PREPARED BY.....	1
1. Laboratory Session 1.....	3
2. Laboratory Session 2.....	9
3. Laboratory Session 3.....	9
4. Laboratory Session 4.....	9

Laboratory Session 1

INTRODUCTION:

The main objective of this lab session was to understand the benefits of using a version control system and get hands-on experience with using git repositories, and also learning about remote repositories, mainly involving key concepts like pull, push, commit, etc. After this, I have worked on the pylint workflow-based GitHub actions to improve the code quality.

TOOLS USED:

The main tool used for this lab for Git. Since I was a little bit familiar with Git terminal, I used that to execute the Git commands, and to make the remote repository, I used the web-based GitHub, and also used it for the workflows. The pylint workflow was used in GitHub Actions to improve the code quality and statistically analyze the code quality.

SETUP:

Terminal was used to set up Git in my system. The following were the commands used to set it up: (since I had it already set up, I did not use the command `git config --global user.name "gaaurav.budhwani@iitgn.ac.in"` because it would simply show the pop-up of multiple values for which I had to reset all or replace the same.

```
gaauravbudhwani@Gauravs-MacBook-Air ~ % git config --global user.name "Gaurav Budhwani"
gaauravbudhwani@Gauravs-MacBook-Air ~ % git config --global user.email "gaaurav.budhwani@iitgn.ac.in"
gaauravbudhwani@Gauravs-MacBook-Air ~ % git config --global --list
filter.lfs.clean=git-lfs clean -- %f
filter.lfs.smudge=git-lfs smudge -- %f
filter.lfs.process=git-lfs filter-process
filter.lfs.required=true
user.email=gaaurav.budhwani@iitgn.ac.in
user.name=Gaurav Budhwani
core.editor=code --wait
```

After setting up, the verification is done using the `--global --list` command as seen above.

If 2-way authentication is turned on in the GitHub account, then the process also involves putting the password and a 2-digit number that is sent to the mobile app.

The setup is now complete.

METHODOLOGY AND EXECUTION:

The first step was to create an empty directory (no README.md). This was done using the following commands: after making the directory, it was also initialized using the *init* command:

```
gauravbudhwani@Gauravs-MacBook-Air ~ % mkdir CS202_lab_1
gauravbudhwani@Gauravs-MacBook-Air ~ % cd CS202_lab_1
gauravbudhwani@Gauravs-MacBook-Air CS202_lab_1 % git init
Initialized empty Git repository in /Users/gauravbudhwani/CS202_lab_1/.git/
```

After initialization, I started creating the [README.md](#), and then added some content to it and committed the file. The following commands were used (echo for making and adding content to the README file):

```
gauravbudhwani@Gauravs-MacBook-Air CS202_lab_1 % echo "# Sample Git Repo" > README.md
gauravbudhwani@Gauravs-MacBook-Air CS202_lab_1 % git add README.md
```

git add command is used to update the current content to stage the upcoming commits.

```
gauravbudhwani@Gauravs-MacBook-Air CS202_lab_1 % git commit -m "Readme added"
[main (root-commit) 921db38] Readme added
1 file changed, 1 insertion(+)
create mode 100644 README.md
gauravbudhwani@Gauravs-MacBook-Air CS202_lab_1 % git log
commit 921db38504a05f5a650fe41e3f896298caf997c2 (HEAD -> main)
Author: Gaurav Budhwani <gaurav.budhwani@iitgn.ac.in>
Date: Wed Aug 6 00:24:50 2025 +0530

Readme added
```

After adding the changes were finally committed using the *commit -m* command, and the changes were checked using the *log* command, which shows the history of the commits.

The next step is to create a new GitHub repository as shown below:

1

General

Owner *

gaurav-budhwani

/

Repository name *

CS202_lab_1_Remote.

CS202_lab_1_Remote. is available.

Great repository names are short and memorable. How about [curly-palm-tree?](#)

Description

0 / 350 characters

2

Configuration

Choose visibility *

Choose who can see and commit to this repository

Public

Add README

READMEs can be used as longer descriptions. [About READMEs](#)

Off

Add .gitignore

.gitignore tells git which files not to track. [About ignoring files](#)

No .gitignore

Add license

Licenses explain how others can use your code. [About licenses](#)

No license

Create repository

The next thing was to remotely add the local repository to the GitHub web-based repository. This was done using the *remote add* command. The changes were then pushed to the GitHub repository as follows:

```
gauravbudhwani@Gauravs-MacBook-Air CS202_lab_1 % git branch -M main
gauravbudhwani@Gauravs-MacBook-Air CS202_lab_1 % git remote add origin https://github.com/gaurav-budhwani/CS202_lab_1_Remote.git
gauravbudhwani@Gauravs-MacBook-Air CS202_lab_1 % git push -u origin main
```

```
Enumerating objects: 3, done.
Counting objects: 100% (3/3), done.
Writing objects: 100% (3/3), 238 bytes | 238.00 KiB/s, done.
Total 3 (delta 0), reused 0 (delta 0), pack-reused 0
To https://github.com/gaurav-budhwani/CS202_lab_1_Remote.git
 * [new branch]      main -> main
branch 'main' set up to track 'origin/main'.
```

Initially, the Repository was empty, but as soon as the local repository was pushed, the README file was visible.

The next step was to clone the repository (web-based) using the *git clone* command.

```
gauravbudhwani@Gauravs-MacBook-Air clones % git clone https://github.com/gaurav-budhwani/CS202_lab_1_Remote.git
Cloning into 'CS202_lab_1_Remote'...
remote: Enumerating objects: 3, done.
remote: Counting objects: 100% (3/3), done.
remote: Total 3 (delta 0), reused 3 (delta 0), pack-reused 0 (from 0)
Receiving objects: 100% (3/3), done.
```

```
gauravbudhwani@Gauravs-MacBook-Air CS202_lab_1_Remote % git pull origin main

remote: Enumerating objects: 5, done.
remote: Counting objects: 100% (5/5), done.
remote: Compressing objects: 100% (2/2), done.
remote: Total 3 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
Unpacking objects: 100% (3/3), 983 bytes | 245.00 KiB/s, done.
From https://github.com/gaurav-budhwani/CS202_lab_1_Remote
 * branch                main          -> FETCH_HEAD
   921db38..bee48cd       main          -> origin/main
Updating 921db38..bee48cd
Fast-forward
 README.md | 1 +
 1 file changed, 1 insertion(+)
```

As seen above, the repository had only the README file added before.

The next task was to create a python script in the online repository which was done as follows:

```
gauravbudhwani@Gauravs-MacBook-Air CS202_lab_1_Remote % touch calculator.py

gauravbudhwani@Gauravs-MacBook-Air CS202_lab_1_Remote % git add calculator.py
gauravbudhwani@Gauravs-MacBook-Air CS202_lab_1_Remote % git commit -m "Added Python code >30 lines"
[main b5cb83a] Added Python code >30 lines
 1 file changed, 65 insertions(+)
 create mode 100644 calculator.py
gauravbudhwani@Gauravs-MacBook-Air CS202_lab_1_Remote % git push origin main
Enumerating objects: 4, done.
Counting objects: 100% (4/4), done.
Delta compression using up to 10 threads
Compressing objects: 100% (3/3), done.
Writing objects: 100% (3/3), 873 bytes | 873.00 KiB/s, done.
Total 3 (delta 0), reused 0 (delta 0), pack-reused 0
To https://github.com/gaurav-budhwani/CS202_lab_1_Remote.git
 bee48cd..b5cb83a  main -> main
```

As seen above the file [calculator.py](#) was added to the remote repository.

```
gauravbudhwani@Gauravs-MacBook-Air-6 CS202_lab_1 % ls
README.md
```

The local repository had no changes whatsoever. But after using the *git pull* command, the changes were reflected in the local repository as well.


```

gaauravbudhwani@Gauravs-MacBook-Air-6 CS202_lab_1 % git pull
remote: Enumerating objects: 13, done.
remote: Counting objects: 100% (13/13), done.
remote: Compressing objects: 100% (8/8), done.
remote: Total 11 (delta 0), reused 8 (delta 0), pack-reused 0 (from 0)
Unpacking objects: 100% (11/11), 2.40 KiB | 272.00 KiB/s, done.
From https://github.com/gaurav-budhwani/CS202_lab_1_Remote
   921db38..0338d3a  main       -> origin/main
Updating 921db38..0338d3a
Fast-forward
 .github/workflows/pylint.yml | 26 ++++++
 README.md                    |  1 +
 calculator.py                 | 65 ++++++
 3 files changed, 92 insertions(+)
 create mode 100644 .github/workflows/pylint.yml
 create mode 100644 calculator.py

```

The next task was to improve the code quality using pylint:

```

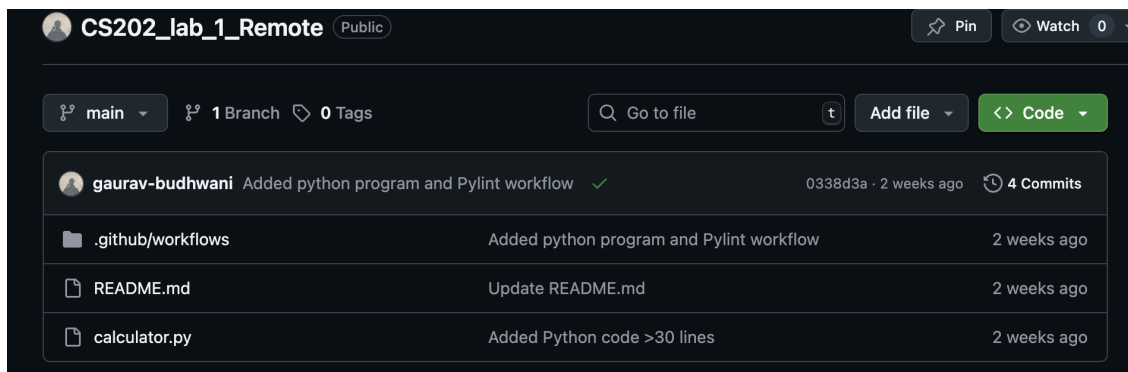
gaauravbudhwani@Gauravs-MacBook-Air CS202_lab_1_Remote % mkdir -p .github/workflows
gaauravbudhwani@Gauravs-MacBook-Air CS202_lab_1_Remote % touch .github/workflows/pylint.yml
gaauravbudhwani@Gauravs-MacBook-Air CS202_lab_1_Remote % open .
gaauravbudhwani@Gauravs-MacBook-Air CS202_lab_1_Remote % git add calculator.py .github/workflows/pylint.yml
gaauravbudhwani@Gauravs-MacBook-Air CS202_lab_1_Remote % git commit -m "Added python program and Pylint workflow"
[main 0338d3a] Added python program and Pylint workflow
 1 file changed, 26 insertions(+)
   create mode 100644 .github/workflows/pylint.yml
gaauravbudhwani@Gauravs-MacBook-Air CS202_lab_1_Remote % git push origin main
Enumerating objects: 6, done.
Counting objects: 100% (6/6), done.
Delta compression using up to 10 threads
Compressing objects: 100% (3/3), done.
Writing objects: 100% (5/5), 662 bytes | 662.00 KiB/s, done.
Total 5 (delta 0), reused 0 (delta 0), pack-reused 0
To https://github.com/gaurav-budhwani/CS202_lab_1_Remote.git
   b5cb83a..0338d3a  main -> main

```

The code was written using VS Code:

```
Users > gauravbudhwani > Documents > clones > CS202_lab_1_Remote > .github > workflows > ! pylint.yml
1  name: Pylint Check
2
3  on:
4    push:
5      branches: [ "main" ]
6    pull_request:
7      branches: [ "main" ]
8
9  jobs:
10   lint:
11     runs-on: ubuntu-latest
12
13     steps:
14       - name: Checkout code
15         uses: actions/checkout@v3
16
17       - name: Set up Python
18         uses: actions/setup-python@v4
19         with:
20           python-version: "3.x"
21
22       - name: Install pylint
23         run: pip install pylint
24
25       - name: Run pylint
26         run: pylint calculator.py
27
```

And after pushing the code the repository showed a tick (green)



CS202_lab_1_Remote (Public)

main 1 Branch 0 Tags

gaurav-budhwani Added python program and Pylint workflow ✓ 0338d3a · 2 weeks ago 4 Commits

File	Commit Message	Time
.github/workflows	Added python program and Pylint workflow	2 weeks ago
README.md	Update README.md	2 weeks ago
calculator.py	Added Python code >30 lines	2 weeks ago

RESULTS AND ANALYSIS:

The main motive was to understand the applications of git. I could use the git commands directly from my terminal instead of the typical interface on the internet. The results of every git command were available to me as soon as it was sent, so there was no lag. In case of any errors, it was specific so it made it easier to fix, whereas the errors in pylint were not that specific, and I had some trouble writing the typical style code of pylint.

DISCUSSIONS AND CONCLUSION:

At its core, this lab session set the basic foundation and understanding of using git in the local environment (not using the internet), it also shows how versatile git is, we can make a local repository, remote access it using the internet, use various commands like pull (get updates from the repo if we are collaborating with someone else), push (send updates), add (making new files), commit and clone. I faced some difficulties in the start when I was setting up the username because I did it before, which made distinct values but since user name and email id has to be a unique identifier, I learned that either we can replace the original or we can reset the current and set new values (the method I went with). Also, while writing the pylint, I noticed the style of code expected was so very different from the typical code I used to write, and the errors were not specific, which made it a lot more difficult to fix the errors. So, Pylint can be improved in terms of code structure and style in the future.

REFERENCES:

- [1] [Lab Assignment 1](#)
- [2] [Git Commands Sheet](#)

Laboratory Session 2

INTRODUCTION:

Software repositories hosted on platforms like GitHub record rich histories of software development through commits. Commit messages are an important method of communication between developers, maintainers, and automated tools as they explain the rationale behind changes in the code. In practice, commit messages are often inaccurate, ambiguous, and incomplete, even with bug-fixing commits where multiple files or changes are batched together. This reduces the maintainability of software projects and makes downstream research tasks like automated program repair and vulnerability localization more difficult. This lab seeks to analyze bug-fixing commits in the wild, examine the precision of developer-generated and machine-generated commit messages, and create a rectifier for the purpose of generating more precise per-file messages. In doing so, the lab will provide a foundation for understanding how developers convey fixes and how tools can support this process.

TOOLS USED:

The analysis was done using the tools mentioned in the Lab document, which include using Python for programming, PyDriller for analysing the repository (extracting commits and diffs), and a Pre-trained LLM to predict the rectified message from file-level diffs. Standard libraries in python were also used for text processing and progress tracking. All these tools together helped in the generation of the required csv database, rectifier to improve precision and helped in overall efficiency of the process.

SETUP:

All of the experiments were done in Python 3.1 (and above) in a linux based system. The machines were equipped with sufficient CPU and memory to handle the large repositories, since we were required to use an virtual machine, I used the one available in lab (oracle vm, SET-IITGN) which made the process much slower, the machine lagged in between and had to be given time to come back from an unresponsive state. Furthermore, GPU was required (optionally) for the LLM interface. The core packages installed included PyDriller for repository mining, Transformers and Torch for model inference, and Pandas for data manipulation.

REPOSITORY SELECTION CRITERIA:

The identify a suitable repository for analysis, I adopted a hierarchical funnel approach inspired by the lecture 2 slides. To find the repository, I used the SEART Github Search

Engine, I first restricted the size to medium-large sized projects with at least 1000 Github stars and 100+ (results). This would ensure that the chosen repository is widely used, actively mentioned, and representative of a real-world problem.

General

Search by keyword in name

Contains

Language

License

Has topic

Uses Label

History and Activity

Number of Commits

1500

2500

Number of Contributors

min

max

Number of Issues

500

max

Number of Pull Requests

min

max

Number of Branches

min

max

Number of Releases

min

max

Popularity Filters

Number of Stars

1000

max

Number of Watchers

1000

max

Number of Forks

min

max

Size of codebase

Non Blank Lines

min

max

Code Lines

min

max

Comment Lines

min

max

Date-based Filters

Created Between

dd/mm/yyyy

dd/mm/yyyy

Last Commit Between

dd/mm/yyyy

dd/mm/yyyy

Additional Filters

Sorting

Name

Ascending

Repository Characteristics

Exclude Forks

Only Forks

Has Wiki

Has License

Has Open Issues

Has Pull Requests

< Back

JS

airbnb/javascript

< < 1 > >

Jump to page...

Go!

JS

airbnb/javascript

Commits: 1983

👁 Watchers: 3757

☆ Stars: 145599

🍴 Forks: 26559

🔗 Total Issues: 1341

🔗 Total Pull Req: 1611

🔗 Branches: 5

👤 Contributors: 402

🔴 Open Issues: 95

🔴 Open Pull Req: 71

📦 Releases: 0

📦 Size: 3.6 KB

⏰ Created: 2012-11-01

🕒 Updated: 2024-12-09

📅 Last Push: 2024-12-09

📅 Last Commit: 2024-12-07

📄 Code Lines: 7,188

📄 Comment Lines: 965

Blank Lines: 1,983

Last Commit SHA: [11f986fdc7d6b4c80e396437e9c45c939362bdee](#)

Show More

This repository was chosen for the analysis. (Huge plus point is that the last commit is of 5 days ago, which means it's still in work and active).

```
# repo
repo = "https://github.com/airbnb/javascript"
keywords = [
    "fix", "bug", "issue", "resolve", "bugfix", "fixes", "fixed",
    "crash", "solves", "testcase", "fail", "except",
    "overflow", "break", "workaround", "stop", "failure",
    "crash", "freez", "problem", " problem", "avoid", "break",
    " break", "issue", " issue", "error", "test", "hang"
]
```

First, we'll mention all the keywords we are looking for in an array and also define the repo.

```
# commits and diffs
commits = []
diffs = []

for c in Repository(repo).traverse_commits():
    if any(i in c.msg.lower() for i in keywords):
        commits.append({
            "hash": c.hash,
            "message": c.msg,
            "hashes_of_parents": [h for h in c.parents],
            "is_merge_commit": len(c.parents) > 1,
            "list_of_modified_files": [m.filename for m in c.modified_files]
        })
        for m in c.modified_files:
            diffs.append({
                "hash": c.hash,
                "message": c.msg,
                "filename": m.filename,
                "source_code_before": m.source_code_before,
                "source_code_after": m.source_code,
                "diff": m.diff
            })

bugfix = pd.DataFrame(commits)
bugfix.to_csv("bugfix.csv", index=False)

diff = pd.DataFrame(diffs)
diff.to_csv("diff.csv", index=False)

print(f"Saved {len(bugfix)} bugfix commits and {len(diff)} diffs")
```

We'll then extract the commits and diffs into an array (initially) and then convert that into a dataframe and finally save it into a comma-separated file (.csv). For extracting the commits as well as the diffs, we are using the Repository tool available inside the pydriller library, at its core, we are searching the keywords in each of the commits and if one is found, we are appending its features (hash, message, hashes of its parents, is a merge commit and list of modified files into the commit array defined earlier and then using that we are extracting the diffs (using the same commit whose keyword is matched).

A final message is shown to view all of the commits and diffs found within the repository for the given number of keywords defined.

```

# manual rectifier
def my_rectifier(filename, diff, org_msg):
    fname = filename.lower()
    diff_t = str(diff).lower()

    if "test" in fname or "assert" in diff_t:
        return f"Update tests in {filename}"
    elif fname.endswith(".py"):
        if any(i in org_msg.lower() for i in ["fix", "bug"]):
            return f"Fix bug in {filename}"
        else:
            return f"Modify Python code in {filename}"
    elif fname.endswith(".md") or fname.endswith(".rst"):
        return f"Update documentation in {filename}"
    elif fname.endswith(".html") or fname.endswith(".css"):
        return f"Update UI layout in {filename}"
    else:
        return f"Update {filename}"

```

The assignment had a major task for building our own rectifier, and since this was a open ended thing to do, the first thing I tried was making a simple rectifier which will check just the defined keywords and make a suggestion according to the name of the file and not on the contents of the file. This is a very basic rectifier and a better one can be made which can take into account the message developer made and the message which was inferred by the LLM and then make a rectified message which can combine all the good qualities of the developer's commit and the LLM's inference.

```

# LLM
print("Loading LLM model (this may take a while)...")
model_name = "mamiksik/CommitPredictorT5"
tokenizer = AutoTokenizer.from_pretrained(model_name)
model = AutoModelForSeq2SeqLM.from_pretrained(model_name)

def llm_predict(diff_text):
    if not diff_text or diff_text.strip() == "":
        return "No changes"
    inputs = tokenizer(diff_text, return_tensors="pt", truncation=True, max_length=512)
    outputs = model.generate(**inputs, max_length=64)
    return tokenizer.decode(outputs[0], skip_special_tokens=True)

# add to the database
df = pd.read_csv("diff.csv")

df["LLM Inference (fix type)"] = df["diff"].apply(lambda d: llm_predict(str(d)))
df["Rectified Message"] = df.apply(
    lambda row: my_rectifier(
        row.get("filename", ""),
        row.get("diff", ""),
        row.get("message", ""),
    ), axis=1
)

# save
df.to_csv("diff_rectified.csv", index=False)

print("Final dataset saved as diff_rectified.csv")
print("Columns: hash, message (developer), filename, source_code_before, source_code_after, diff, llm_message, rectified_message")

```

RESULTS AND ANALYSIS:

Making a virtual environment so that the other libraries cannot interfere with the program.

Running the python program for mining the repository.

Final dataset with all the columns needed is saved.

hash	message	filename	diff	LLM Inference (fix type)	Rectified Message
8cef7f1ec6d6cd1a0e0e5c7915d4a001d3cb10e2	fix the baker link	README.md	<pre>@@ -1246,7 +1246,7 @@ ## Contributors - [Dave Augustine](//github.com/daveaugustine) [@daveaugustine](//twitter.com/daveaugustine) - - [Matt Baker](//github.com/reissbaker) (@reissbaker)(//twitter/reissbaker) + - [Matt Baker](//github.com/reissbaker) [@reissbaker](//twitter/reissbaker) ## License</pre>	add missing missing tag [ci skip]	Update documentation in README.md

ab5910ae9ad13acde bd7f104acab8e63fa2 fcdd	braces for multit-line blocks only. optional for single-line blocks. fixes #3	README.md	@@ -588,10 +588,14 @@ ## Blocks - - Use braces with blocks and always on multiple lines. + - Use braces with all multi-line blocks. ```:javascript // bad + if (test) + return false; + + // good + if (test) return false; // good	add missing semicolon	Update documentation in README.md
---	---	-----------	--	--------------------------	--------------------------------------

As can be seen the LLM suggests a inference like (add missing missing tag and add missing semicolon) but the my simple rectifier just names the file and some keywords in the commit, for example (update documentation in file_name, etc) which can actually be better sometimes by just mentioning the file name and then seeing the commit, but for situations where in we have search for a particular change LLM is better.

EVALUATION SCORES:

```
df = pd.read_csv("diff_rectified.csv")

sample_size = 50
sample_df = df.sample(n=sample_size, random_state=42)

ann_file = "annotations.csv"

if not os.path.exists(ann_file):
    # Create a fresh template
    template = sample_df[["hash", "filename", "message", "LLM Inference (fix type)", "Rectified Message"]].copy()
    template["dev_score"] = "" # to be filled manually (0/1/2)
    template["lm_score"] = ""
    template["rectified_score"] = ""
    template["notes"] = ""
    template.to_csv(ann_file, index=False)
    print(f"Annotation template saved as {ann_file}. Open it in Excel/Sheets and score commits (0/1/2).")
    exit()

ann = pd.read_csv(ann_file)

def hit_rate(scores):
    return (scores == 2).mean()

def partial_rate(scores):
    return (scores == 1).mean()

def ci_bootstrap(scores, n=2000, alpha=0.05):
    arr = np.array(scores == 2, dtype=float)
    boots = [resample(arr, replace=True).mean() for _ in range(n)]
    lo, hi = np.percentile(boots, [100*alpha/2, 100*(1-alpha/2)])
    return float(lo), float(hi)

for col, label in [
    ("dev_score", "Developer"),
    ("lm_score", "LLM"),
    ("rectified_score", "Rectifier")]:
    if col not in ann.columns:
        continue
    vals = pd.to_numeric(ann[col], errors="coerce").dropna().astype(int)
    if len(vals) == 0:
        continue
    hr = hit_rate(vals)
    pr = partial_rate(vals)
    lo, hi = ci_bootstrap(vals)
    print(f"{label}: hit rate={hr:.2%} (95% CI {lo:.2%}-{hi:.2%}), partial={pr:.2%}")
```


The above was used to generate a sample from the diff file and I manually then added scores according to my understanding of the code, Now the question arises to why not automating it and the answer is simply because “precision” means “does this message really describe *what* + *where* in the diff,” a human typically needs to check it. Automating will only approximate, and you’ll lose nuance. The scores allotted were (2: “Precise”, 1: “Partial” and 0: “vague/incorrect”)

The following is the table generated:

hash	filename	message	LLM Inference (fix type)	Rectified Message	dev_score	llm_score	rectified_score
52d3401f953 afcf09ca64db bb177c62848 32aae6	strict.js	[eslint config] [eslint config base] [breaking] Migrate non-React rules to a separate linter config.	nan return null	Update strict.js	2	0	1
7ff6303513b4 6e8b7dd0bc0 3327b138a6d b7b3b4	README.md	[guide] Use HTTPS for links to ESLint documentation The result of a `s_http://eslint \.org_https://e slint\.org_g`. Avoids the HTTP 302s of the non-HTTPs URLs.	add missing link to lint config	Update documentation in README.md	2	2	1

The above is only a few samples; this was done for 50 samples over time, and the following is the evaluation obtained:

```
(.venv) gauravbudhwani@Gauravs-MacBook-Air Lab2 % python3 partf.py
Annotation template saved as annotations.csv. Open it in Excel/Sheets and score commits (0/1/2).
(.venv) gauravbudhwani@Gauravs-MacBook-Air Lab2 % python3 partf.py
Developer: hit rate=76.00% (95% CI 64.00%-88.00%), partial=24.00%
LLM: hit rate=50.00% (95% CI 36.00%-64.00%), partial=32.00%
Rectifier: hit rate=12.00% (95% CI 4.00%-22.00%), partial=88.00%
```

As expected the simple rectifier's (manual) has the lowest hit rate and surprisingly, according to my analysis the hit rate of the developer's commits have the highest hit rate, this is due to detailed commits performed by them which is then shortened and sometimes misunderstood by the LLM and that is why if we want to make a better rectified message we should use more powerful LLM on the previously inferred message and the developer's commit as well. It is important to note that sometimes the developer's message is not precise but the LLM infers better than the developer, so we see all types of cases here (same goes for manual rectifier).

CONCLUSION:

Though I initially faced difficulties in understanding the actual goal of the lab, and since we were not allowed to use minecraft or minecpp tools, the code had to be written from scratch. The main task of difficulty was how to write our own algorithm of rectifier for the commit messages of the developer, one idea I had was to use the LLM inference and the developer's commit which may not be accurate everytime and then feed the combination of this and some other parameters to another LLM which may be able to infer better than the LLM itself. One more thing that can be done is use diff to actually see what has changed and suggest a commit based on that using the LLMs available.

References:

[1]  CS202_Lecture_2_05_Aug_2025.pdf

[2] [Pydriller Documentation](#)

Laboratory Session 3

INTRODUCTION

This lab is an extension of the previous lab, wherein we had to extract the bug-fix dataset using Pydriller with additional data from the LLM and our own build rectifier. In this lab, the main focus will be on additional metrics like the Maintainability Index, Cyclomatic complexity, and Lines of Code for each bug fix before and after the change. The tool that will be used in this lab will be radon. The metrics generated from this will then be compared with Semantic and Token based similarity measures to investigate the relationship between magnitude of changes and the code quality.

TOOLS USED

The main tool used in this lab was Radon, which was applied to compute structural metrics such as Maintainability Index (MI), Cyclomatic Complexity (CC), and Lines of Code (LOC) before and after bug-fix commits. For computing change magnitude metrics, the CodeBERT model (from Hugging Face) was employed to calculate semantic similarity, while SacreBLEU was used to measure token-level similarity. The initial dataset was obtained from the Lab 2 commit-level analysis, and all computations were performed using Python 3 within a virtual environment. For handling data and storing results, libraries such as Pandas were used.

SETUP

For the desktop available in the lab (Linux virtual machine) and Windows. I created a virtual machine environment using the following commands, which ensured that the installed packages do not interfere with the program (gives the program some isolation to run in a separate environment).

```
PS C:\cs202\lab3> python -m venv venv  
PS C:\cs202\lab3> .\venv\Scripts\activate
```

Libraries were installed using the last command provided to the terminal.
All of this was done inside the designated folder for this lab session.

```
(venv) PS C:\cs202\lab3> pip install pandas sacrebleu torch radon transformers
Collecting pandas
  Downloading pandas-2.3.2-cp312-cp312-win_amd64.whl.metadata (19 kB)
Collecting sacrebleu
  Downloading sacrebleu-2.5.1-py3-none-any.whl.metadata (51 kB)
Collecting torch
  Downloading torch-2.8.0-cp312-cp312-win_amd64.whl.metadata (30 kB)
Collecting radon
  Downloading radon-6.0.1-py2.py3-none-any.whl.metadata (8.2 kB)
Collecting transformers
  Downloading transformers-4.56.1-py3-none-any.whl.metadata (42 kB)
Collecting numpy>=1.26.0 (from pandas)
  Downloading numpy-2.3.2-cp312-cp312-win_amd64.whl.metadata (60 kB)
Collecting python-dateutil>=2.8.2 (from pandas)
  Using cached python_dateutil-2.9.0.post0-py2.py3-none-any.whl.metadata (8.4 kB)
```

PROCESS AND EXECUTION

The repository data that was used initially was the output file that was generated in the previous lab, but it is important to note that radon only works on python scripts and the repository that I used in previous lab uses javascript (majorly), So, for this lab I had to use a different repository and for that I searched the following constraints on SEART:

General

python

Contains ▾

License

Has topic

Language

Uses Label

History and Activity

Number of Commits

1000

max

Number of Contributors

min

max

Number of Issues

500

max

Number of Pull Requests

min

max

Number of Branches

min

max

Number of Releases

min

max

Date-based Filters

Created Between

dd/mm/yyyy

▢

dd/mm/yyyy

▢

Last Commit Between

dd/mm/yyyy

▢

dd/mm/yyyy

▢

Popularity Filters

Number of Stars

1000

max

Size of codebase ⓘ

Non Blank Lines

min

max

Number of Watchers

min

max

Code Lines

min

max

Number of Forks

min

max

Comment Lines

min

max

Additional Filters

Sorting

Name ▾

Ascending ▾

Repository Characteristics

☐ Exclude Forks

☐ Only Forks

☐ Has Wiki

☐ Has License

☐ Has Open Issues

☐ Has Pull Requests

Search


[openai/openai-python](https://github.com/openai/openai-python)
🔗

Commits: 1101	👁 Watchers: 345	☆ Stars: 28568	🍴 Forks: 4240
🕒 Total Issues: 1090	🔗 Total Pull Req: 1363	🌿 Branches: 16	👤 Contributors: 136
🔴 Open Issues: 92	🔗 Open Pull Req: 52	📦 Releases: 328	📄 Size: 6.17 KB
+ Created: 2020-10-25	🕒 Updated: 2025-09-04	↑ Last Push: 2025-09-04	📅 Last Commit: 2025-09-04
<> Code Lines: 63,423	🗨 Comment Lines: 23,329	Blank Lines: 23,764	

Last Commit SHA: [2ad11112988e998fcf5adb805bae38501d22318](#)

[Show More](#)

The above GitHub repository was used to for further analysis.

The following is the code that was used for this lab.

```
def safe_str(x):
    if pd.isna(x):
        return ""
    return str(x)

def compute_radon_metrics(code):
    try:
        if not code or code.strip() == "":
            return (np.nan, np.nan, np.nan)
        mi_score = mi_visit(code, True)
        cc_blocks = cc_visit(code)
        cc_sum = sum([b.complexity for b in cc_blocks]) if cc_blocks else 0.0
        raw = raw_analyze(code)
        sloc = raw.sloc
        return (float(mi_score), float(cc_sum), int(sloc))
    except Exception:
        return (np.nan, np.nan, np.nan)

def bleu_score(reference, hypothesis):
    try:
        if safe_str(reference).strip() == "" or safe_str(hypothesis).strip() == "":
            return np.nan
        sc = sacrebleu.sentence_bleu(hypothesis, [reference])
        return float(sc.score) / 100.0
    except Exception:
        return np.nan
```

Here, I am first checking for a NAN value (on input), and if found, then safely converting it to a string (done after facing errors). The next function computes the metrics using the radon tool, and the BLEU score is also computed.

```

class CodeBERTBatchEmbedder:
    def __init__(self, model_name="microsoft/codebert-base", device=None):
        self.device = device or ("cuda" if torch.cuda.is_available() else "cpu")
        print(f"[CodeBERT] device: {self.device}")
        self.tokenizer = AutoTokenizer.from_pretrained(model_name)
        self.model = AutoModel.from_pretrained(model_name).to(self.device)
        self.model.eval()

    def _mean_pool(self, last_hidden_state, attention_mask):
        attn = attention_mask.unsqueeze(-1)
        summed = (last_hidden_state * attn).sum(dim=1)
        counts = attn.sum(dim=1).clamp(min=1)
        return (summed / counts).cpu().numpy()

    def embed_batch(self, texts, max_length=512, batch_size=32):
        """
        texts: list[str]
        returns: list[np.array] (None for empty strings)
        """
        out = [None] * len(texts)
        # indices with non-empty text
        nonempty = [i for i, t in enumerate(texts) if t and t.strip() != ""]
        for i in range(0, len(nonempty), batch_size):
            batch_idx = nonempty[i:i+batch_size]
            batch_texts = [texts[j] for j in batch_idx]
            toks = self.tokenizer(batch_texts, truncation=True, padding=True, max_length=max_length, return_tensors="pt")
            input_ids = toks["input_ids"].to(self.device)
            attn = toks["attention_mask"].to(self.device)
            with torch.no_grad():
                res = self.model(input_ids=input_ids, attention_mask=attn)
                last_hidden = res.last_hidden_state
                pooled = self._mean_pool(last_hidden, attn) # shape (B, hidden)
                for k, idx in enumerate(batch_idx):
                    out[idx] = pooled[k]
        return out

def cosine_sim(a, b):
    if a is None or b is None:
        return np.nan
    a = np.asarray(a).reshape(1, -1)
    b = np.asarray(b).reshape(1, -1)
    return float(cosine_similarity(a, b)[0][0])

```

A class wrapper around CodeBERT, a pre-trained transformer for code.

The `__init__` function loads the tokenizer and model on CPU/GPU

`__mean_pool__` averages the hidden states to create a single embedding vector per snippet

`embed_batch` embeds a batch of code snippets at once (faster than row-by-row, tried and then changed to this method)

```

def main(args):
    df = pd.read_csv(args.input)
    print(f"Loaded {len(df)} rows from {args.input}")
    df = prepare_columns(df)
    print("\n=== Baseline Descriptive Statistics ===")
    total_commits = df["hash"].nunique() if "hash" in df.columns else np.nan
    total_files = df["filename"].nunique() if "filename" in df.columns else np.nan
    avg_files_per_commit = (
        df.groupby("hash")["filename"].nunique().mean()
        if "hash" in df.columns and "filename" in df.columns else np.nan
    )

    print("Total commits:", total_commits)
    print("Total files:", total_files)
    print("Avg. modified files per commit:", avg_files_per_commit)

    if "LLM Inference (fix type)" in df.columns:
        print("\nFix type distribution:")
        print(df["LLM Inference (fix type)"].value_counts().head(10)) # top 10 only

    if "filename" in df.columns:
        print("\nTop filenames:")
        print(df["filename"].value_counts().head(10))

        df["extension"] = df["filename"].astype(str).apply(
            lambda x: x.split(".")[-1] if "." in x else ""
        )
        print("\nTop file extensions:")
        print(df["extension"].value_counts().head(10))
    print("=" * 35)

```

In the above code, the baseline statistics are computed (total commits, average modified files per commit). I have also tried to show top fix types, filenames, and extensions.

```

total_commits = df["hash"].nunique() if "hash" in df.columns else np.nan
total_files = df["filename"].nunique() if "filename" in df.columns else np.nan
avg_files_per_commit = df.groupby("hash")["filename"].nunique().mean() if "hash" in df.columns and "filename" in df.columns else np.nan
print("Total commits:", total_commits)
print("Total files:", total_files)
print("Avg files/commit:", avg_files_per_commit)

# compute radon metrics
use_radon_mask = df["filename"].astype(str).str.endswith(".py") | args.force_radon
if args.force_radon:
    print("Force radon for non-.py files enabled.")

# compute radon metrics for rows
for idx in tqdm(df.index, desc="Radon (structural)"):
    if use_radon_mask.iloc[df.index.get_loc(idx)]:
        before = safe_str(df.at[idx, "source_code_before"]) if "source_code_before" in df.columns else ""
        after = safe_str(df.at[idx, "source_code_after"]) if "source_code_after" in df.columns else ""
        mi_b, cc_b, loc_b = compute_radon_metrics(before)
        mi_a, cc_a, loc_a = compute_radon_metrics(after)
    else:
        mi_b = mi_a = cc_b = cc_a = loc_b = loc_a = np.nan
    df.at[idx, "MI_Before"] = mi_b
    df.at[idx, "MI_After"] = mi_a
    df.at[idx, "CC_Before"] = cc_b
    df.at[idx, "CC_After"] = cc_a
    df.at[idx, "LOC_Before"] = loc_b
    df.at[idx, "LOC_After"] = loc_a
    # changes
    df.at[idx, "MI_Change"] = (mi_a - mi_b) if (not pd.isna(mi_a) and not pd.isna(mi_b)) else np.nan
    df.at[idx, "CC_Change"] = (cc_a - cc_b) if (not pd.isna(cc_a) and not pd.isna(cc_b)) else np.nan
    df.at[idx, "LOC_Change"] = (loc_a - loc_b) if (not pd.isna(loc_a) and not pd.isna(loc_b)) else np.nan

```

Structural metrics were computed using Radon for Python files, quantifying maintainability, complexity, and size differences.

```
print("Computing token similarity (BLEU)...")
for idx in tqdm(df.index, desc="BLEU"):
    before = safe_str(df.at[idx, "source_code_before"]) if "source_code_before" in df.columns else ""
    after = safe_str(df.at[idx, "source_code_after"]) if "source_code_after" in df.columns else ""
    df.at[idx, "Token_Similarity"] = bleu_score(before, after)

# Semantic similarity (batch embed)
if not args.skip_semantic:
    print("Preparing CodeBERT embedder...")
    embedder = CodeBERTBatchEmbedder()
    # collect unique texts (to avoid repeated embedding)
    texts = []
    idx_map_before = []
    idx_map_after = []
    for idx in df.index:
        before = safe_str(df.at[idx, "source_code_before"]) if "source_code_before" in df.columns else ""
        after = safe_str(df.at[idx, "source_code_after"]) if "source_code_after" in df.columns else ""
        texts.append(before)
        texts.append(after)
        idx_map_before.append(len(texts)-2)
        idx_map_after.append(len(texts)-1)
    # unique mapping
    unique_texts, inverse_idx = np.unique(np.array(texts, dtype=object), return_inverse=True)
    unique_texts = unique_texts.tolist()
    print(f"Unique code snippets to embed: {len(unique_texts)}")
    # embed in batches
    embeddings = embedder.embed_batch(unique_texts, batch_size=args.batch_size)
    # map back
    for row_i, idx in enumerate(df.index):
        emb_b = embeddings[inverse_idx[2*row_i]]
        emb_a = embeddings[inverse_idx[2*row_i + 1]]
        df.at[idx, "Semantic_Similarity"] = cosine_sim(emb_b, emb_a)
```

Token similarity was evaluated using BLEU to identify surface-level textual overlap between original and rectified code. CodeBERT embeddings and cosine similarity were applied to capture meaning-level changes in code.


```

print("Classifying and marking agreement...")
for idx in tqdm(df.index, desc="Classify"):
    sem_sim = df.at[idx, "Semantic_Similarity"]
    tok_sim = df.at[idx, "Token_Similarity"]
    sem_class = None
    tok_class = None
    if not pd.isna(sem_sim):
        sem_class = "Minor" if sem_sim >= args.semantic_threshold else "Major"
    if not pd.isna(tok_sim):
        tok_class = "Minor" if tok_sim >= args.token_threshold else "Major"
    df.at[idx, "Semantic_Class"] = sem_class
    df.at[idx, "Token_Class"] = tok_class
    if sem_class is not None and tok_class is not None:
        df.at[idx, "Classes_Agree"] = "YES" if sem_class == tok_class else "NO"
    else:
        df.at[idx, "Classes_Agree"] = None

# save
df.to_csv(args.output, index=False)
print(f"Wrote output to {args.output}")

```

Threshold based classification was used to separate major and minor fixes, with an agreement check to compare token-based and semantic-based results. The final dataset, containing baseline statistics, structural metrics, semantic and token similarities, and classifications, was exported.

RESULTS AND ANALYSIS

```

(venv) PS C:\cs202\lab3> python lab3_metrics.py -i diff_rectified.csv -o diff_rectified_with_metrics.csv --batch-size 64
Loaded 2304 rows from diff_rectified.csv

=== Baseline Descriptive Statistics ===
Total commits: 368
Total files: 651
Avg. modified files per commit: 5.763586956521739

Fix type distribution:
LLM Inference (fix type)
add missing import                210
add missing docstrings            140
add missing docstring             133
add missing comment               117
add missing comments              71
add test for                      34
add missing imports               32
update version number for missing release 19
nan return null                   17
add new example                   17
Name: count, dtype: int64

Top filenames:
filename
__init__.py          141
README.md            48
completions.py       44
_base_client.py      31
.stats.yml           30
test_completions.py  29
api_requestor.py     29
version.py           28
test_client.py       26
pyproject.toml       26
Name: count, dtype: int64

```

The above terminal screenshot shows that the dataset was loaded successfully and the baseline statistics were calculated.

```
Top filenames:
filename
__init__.py      141
README.md        48
completions.py   44
_base_client.py  31
.stats.yml       30
test_completions.py 29
api_requestor.py 29
version.py       28
test_client.py   26
pyproject.toml   26
Name: count, dtype: int64
```

The above figure shows the most frequent file extensions in the commits of the repository (airbnb/javascript) that were changed the most.

Here, the structural metrics were computed, and now the CodeBert model was used to further compute the semantic and token similarities as follows:

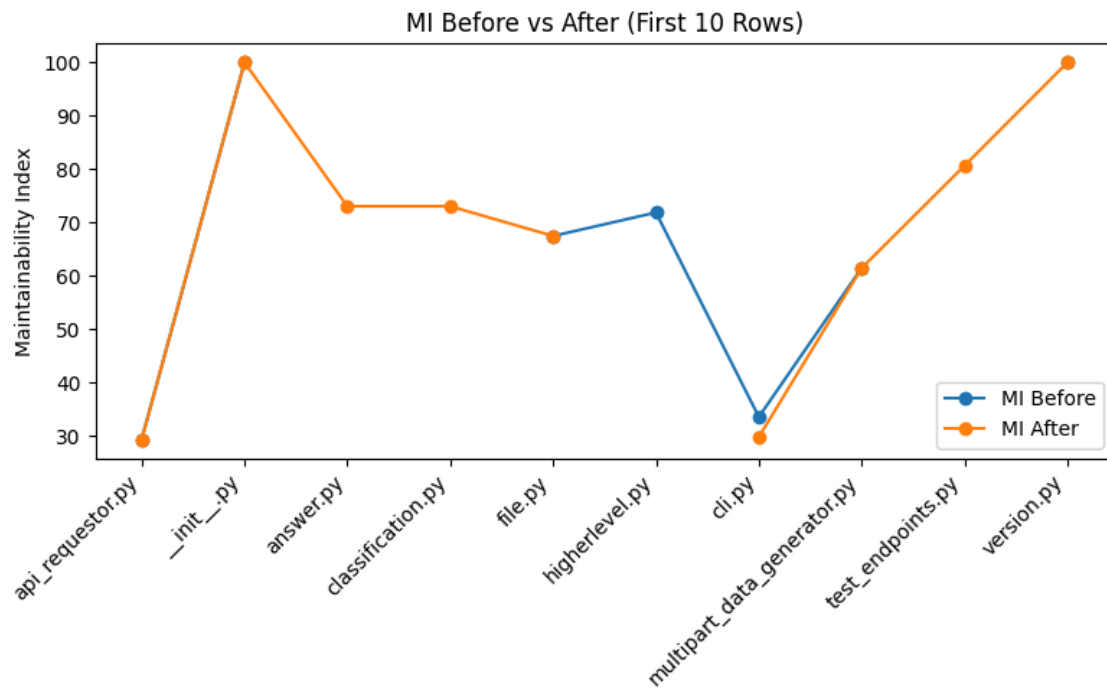
```
Radon (structural): 100%| 2304/2304 [01:59<00:00, 19.35it/s]
Computing token similarity (BLEU)...
BLEU: 100%| 2304/2304 [00:28<00:00, 80.17it/s]
Preparing CodeBERT embedder...
[CodeBERT] device: cpu
Unique code snippets to embed: 3102
Classifying and marking agreement...
Classify: 100%| 2304/2304 [00:00<00:00, 11845.16it/s]
Wrote output to diff_rectified_with_metrics.csv
```

The program completed with zero errors.

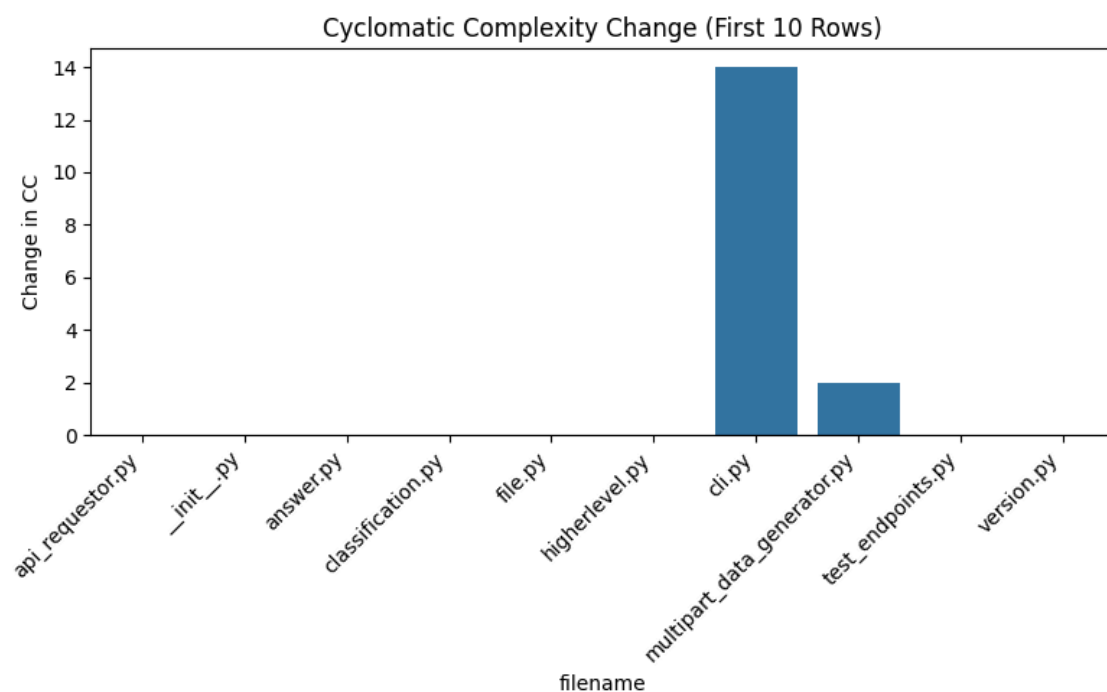
I have used Google Colab for further analysis of the results.

The following are the first 10 rows of the dataset with metrics computed:

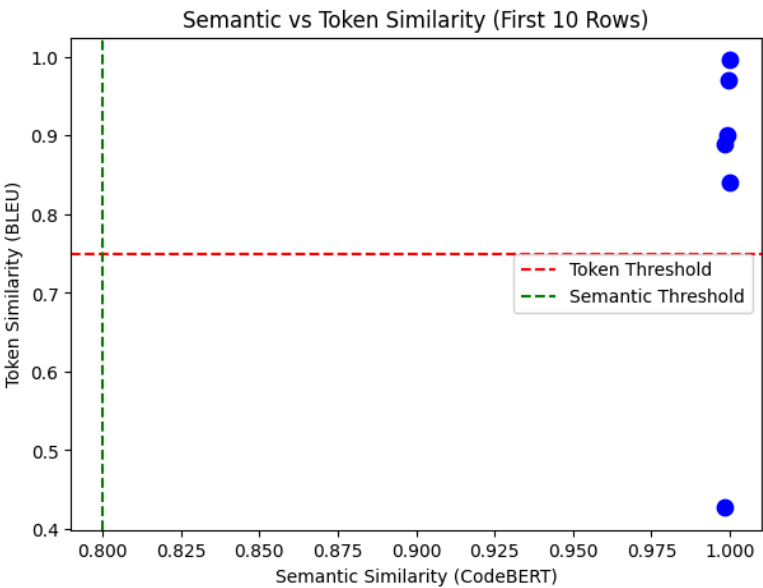
	filename	MI_Before	MI_After	MI_Change	CC_Change	LOC_Change	Semantic_Similarity	Token_Similarity	Semantic_Class	Token_Class	Classes_Agree
0	api_requestor.py	29.348725	29.348725	0.000000	0.0	0.0	1.000000	0.996093	Minor	Minor	YES
1	__init__.py	100.000000	100.000000	0.000000	0.0	1.0	0.999268	0.899845	Minor	Minor	YES
2	answer.py	NaN	73.041182	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN
3	classification.py	NaN	73.041182	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN
4	file.py	67.472677	67.472677	0.000000	0.0	-2.0	0.999635	0.970930	Minor	Minor	YES
5	higherlevel.py	71.869238	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN
6	cli.py	33.552383	29.791687	-3.760696	14.0	76.0	1.000000	0.839968	Minor	Minor	YES
7	multipart_data_generator.py	61.480999	61.461450	-0.019549	2.0	5.0	0.998206	0.889765	Minor	Minor	YES
8	test_endpoints.py	NaN	80.674539	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN
9	version.py	100.000000	100.000000	0.000000	0.0	0.0	0.998409	0.427287	Minor	Major	NO



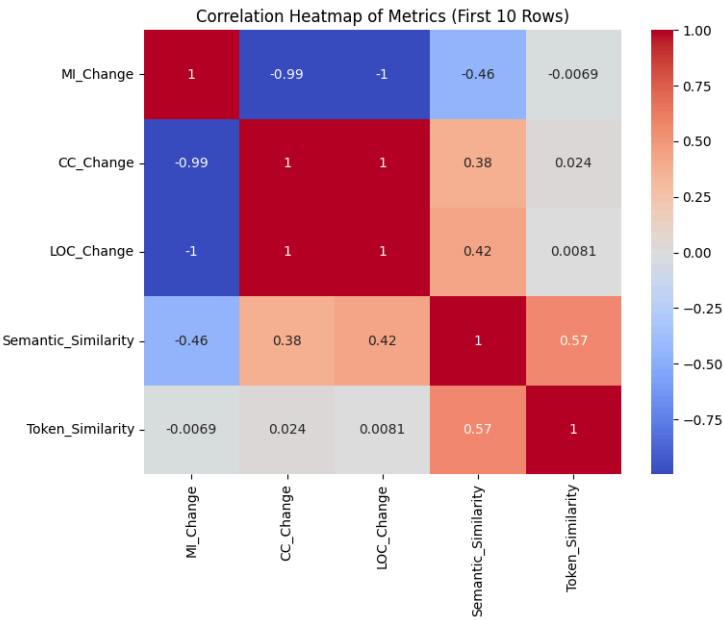
The above figure shows the MI for various files (before and after), and a lot of overlap can be seen. For some files MI increased significantly, and for a few it decreased significantly. In general, refactoring tends to improve maintainability, but not uniformly across all files. Some fixes might prioritize functionality over readability.



Fixes generally did not alter program logic, but in some cases (`cli.py`) fixes introduced new conditional paths, potentially increasing maintenance difficulty.



Token- and semantic-based similarity mostly agree, but cases exist where token similarity underestimates code equivalence. This highlights the importance of semantic analysis. To further see this a correlation plot can be made.



Strong correlation among code change metrics (MI, CC, LOC), while semantic and token similarity are moderately correlated with each other but weakly with code metrics. (2 Groups formed here)

The above analysis is mostly the same for the complete dataset, but the plots become more congested, and thus, it is shown and explained for only the first 10 rows. However, all the plots can be seen in the Jupyter notebook.

CONCLUSION

The analysis of the first ten rectified commits demonstrates that most fixes improved or maintained the Maintainability Index, with only a few cases showing reduced maintainability due to added complexity. Cyclomatic Complexity generally remained unchanged, except in isolated cases where new logic increased branching. Token and semantic similarity measures largely agreed in classifying fixes as Minor, although semantic analysis proved more reliable in capturing meaning-preserving changes that token-level metrics flagged as major.

REFERENCES

- [1] [Lab 3 Document](#)
- [2] [Radon Documentation](#)
- [3] [CodeBert](#)
- [4] [SacreBLEU](#)

Laboratory Session 4

INTRODUCTION

The main objective of this lab session was to gain exposure to PyDriller, a tool that can be used for mining various GitHub repositories. This tool provides valuable insights and information about the commit history of the repositories, such as the commit SHA, the files that were changed in that commit, the part of the file that was changed (before and after state of the file), as well as the specific differences between the before state and the after state. The lab session also required us to use different algorithms that are used in the git framework for getting the differences; the methods/algorithms are namely the Myers algorithm (with git diff) and the Histogram algorithm (with git diff).

TOOLS USED

The most important tool used in this lab was PyDriller, which was used to obtain the complete information about the commit data. Three repositories were selected using the SEART search engine for filtering repositories based on our requirements (medium to large in this lab). Most of the part of the code was done on a Linux VM, and the analysis was done on macOS (using a virtual environment). For further analysis of the obtained results, various Python libraries such as Pandas, NumPy, and Matplotlib were used.

SETUP

For the desktop available in the lab (Linux virtual machine) and macOS (also Linux). I created a virtual machine environment using the following commands, which ensured that the installed packages do not interfere with the program (gives the program some isolation to run in a separate environment).

```
gauravbudhwani@Gauravs-MacBook-Air ~ % cd Downloads
gauravbudhwani@Gauravs-MacBook-Air Downloads % mkdir -p ~/cs202-lab4
gauravbudhwani@Gauravs-MacBook-Air Downloads % cd ~/cs202-lab4
gauravbudhwani@Gauravs-MacBook-Air cs202-lab4 % python3 -m venv .venv
gauravbudhwani@Gauravs-MacBook-Air cs202-lab4 % source .venv/bin/activate
(.venv) gauravbudhwani@Gauravs-MacBook-Air cs202-lab4 % pip install pydriller gitpython pandas matplotlib tqdm
```

Libraries were installed using the last command provided to the terminal. All of this was done inside the designated folder for this lab session.

PROCESS AND EXECUTION

The first step was to search for a medium to large-sized GitHub repository that could be used for this analysis. One thing to keep in mind was to select real real-world problem

repository so that we can get some nice results. I have searched for repositories with more than 2000 commits and that have more issues opened and are active. The following are the repositories and the selection criteria.

General

Search by keyword in name

Contains

Language

License

Has topic

Uses Label

History and Activity

Number of Commits

1500

2500

Number of Contributors

min

max

Number of Issues

500

max

Number of Pull Requests

min

max

Number of Branches

min

max

Number of Releases

min

max

Date-based Filters

Created Between

dd/mm/yyyy

dd/mm/yyyy

Last Commit Between

dd/mm/yyyy

dd/mm/yyyy

Popularity Filters

Number of Stars

1000

max

Size of codebase

Non Blank Lines

min

max

Code Lines

min

max

Comment Lines

min

max

Number of Watchers

1000

max

Additional Filters

Sorting

Name

Ascending

Repository Characteristics

☐ Exclude Forks

☐ Has License

☐ Only Forks

☐ Has Open Issues

☐ Has Wiki

☐ Has Pull Requests

Results: 19

< Back

1

Jump to page...

Go!

js airbnb/javascript Commits: 1983 ⌵ Total Issues: 1341 ⌵ Open Issues: 95 + Created: 2012-11-01 <> Code Lines: 7,188 # Last Commit SHA: 11f986fdc7d6b4c80e396437e9c45c939362bdee 👁 Watchers: 3757 🔗 Total Pull Req: 1611 🔗 Open Pull Req: 71 📅 Updated: 2024-12-09 📄 Comment Lines: 965 Blank Lines: 1,983 ☆ Stars: 145599 🌿 Branches: 5 📦 Releases: 0 ⬆ Last Push: 2024-12-09 📅 Last Commit: 2024-12-07 🍴 Forks: 26559 👤 Contributors: 402 📄 Size: 3.6 KB Show More	🔥 alamofire/alamofire Commits: 1657 ⌵ Total Issues: 2801 ⌵ Open Issues: 42 + Created: 2014-07-31 <> Code Lines: 178,693 # Last Commit SHA: 46249e016881b96846025ed8c2e5845a822af3fe 👁 Watchers: 1058 🔗 Total Pull Req: 1127 🔗 Open Pull Req: 9 📅 Updated: 2025-08-30 📄 Comment Lines: 9,118 Blank Lines: 14,173 ☆ Stars: 42070 🌿 Branches: 23 📦 Releases: 102 ⬆ Last Push: 2025-08-30 📅 Last Commit: 2025-05-14 🍴 Forks: 7638 👤 Contributors: 244 📄 Size: 38.16 KB Show More	🏢 alibaba/artas Commits: 2074 ⌵ Total Issues: 2094 ⌵ Open Issues: 372 + Created: 2018-08-29 <> Code Lines: 116,192 # Last Commit SHA: 3c09d4a6860771d5e6251df6760cca3fd679ce4b 👁 Watchers: 1124 🔗 Total Pull Req: 864 🔗 Open Pull Req: 88 📅 Updated: 2025-08-30 📄 Comment Lines: 16,727 Blank Lines: 19,804 ☆ Stars: 36662 🌿 Branches: 13 📦 Releases: 59 ⬆ Last Push: 2025-08-29 📅 Last Commit: 2025-07-27 🍴 Forks: 7574 👤 Contributors: 207 📄 Size: 54.31 KB Show More
---	---	--

(criteria used from funnel diagram, lecture notes)(High number of commits, starts, active issues, active contributions)

```
def check_git_available() -> None:
    try:
        subprocess.run(["git", "--version"], check=True, capture_output=True)
    except Exception as e:
        raise SystemExit("ERROR: git is not installed or not on PATH. Install Xcode Command Line Tools or Git.") from e
```

First, we'll check the availability of git in the system since we'll be extracting the repository through the code itself.

```
def repo_dir_name(owner_repo: str) -> str:
    owner_repo = owner_repo.strip().strip("/")
    owner, name = owner_repo.split("/", 1)
    return f"{owner}_{name}"

def clone_repo(owner_repo: str, base_dir: Path) -> Path:
    """
    Clone the repo if needed. Returns the local path.
    """
    local = base_dir / repo_dir_name(owner_repo)
    if local.exists() and (local / ".git").exists():
        return local
    local.parent.mkdir(parents=True, exist_ok=True)
    url = f"https://github.com/{owner_repo}.git"
    print(f"[clone] {owner_repo} -> {local}")
    # Full clone (history). You may replace with --depth to speed up, but you'll miss older commits.
    subprocess.run(["git", "clone", "--no-tags", url, str(local)], check=True)
    return local
```

The name of the repositories is inside the repos.txt. If the repository already exists, we'll use it; else, we'll initiate cloning the repository from GitHub, which includes the commit history for analysis.

```
def run_git_diff(repo_path: Path, parent: str, commit: str, pathspec: str, algorithm: str) -> str:
    """
    Run git diff for a single file with given algorithm, ignoring whitespace/blank lines, zero context.
    Returns the raw patch text (may be empty).
    """
    cmd = [
        "git", "-C", str(repo_path),
        "diff",
        "-w", "--ignore-blank-lines",
        "--unified=0",
        f"--diff-algorithm={algorithm}",
        parent, commit, "--", pathspec
    ]
    res = subprocess.run(cmd, capture_output=True, text=True, errors="replace")
    # We do not check returncode because git diff returns 1 when differences exist if used with certain flags.
    return res.stdout or ""
```

Running the Git built-in diff algorithm (Myers or histogram) for one file between a commit and its parents (returns a raw patch string).


```
def normalize_patch(patch: str) -> str:
    """
    Normalize a patch by removing header lines and keeping only added/removed markers.
    We also strip trailing spaces and skip empty lines, to be conservative in 'match' judgment.
    """
    norm_lines = []
    for line in patch.splitlines():
        if HEADER_STRIP_RE.match(line):
            continue
        if line.startswith("+") or line.startswith("-"):
            s = line.rstrip()
            if s and s not in {"+", "-"}:
                norm_lines.append(s)
    return "\n".join(norm_lines).strip()
```

Removing diff headers and keeping only meaningful lines. This will ensure that the differences are judged purely based on code differences/changes, not like metadata, filename, or line numbers.

```
def classify_category(path: Optional[str]) -> str:
    """
    Classify file path into Source, Test, README, LICENSE, Other.
    """
    if not path:
        return "Other"
    pname = Path(path)
    name = pname.name.lower()

    if name.startswith("readme"):
        return "README"
    if name.startswith("license") or name in {"copying", "licence"}:
        return "LICENSE"
    if re.search(r"^(^/)(tests?|spec|__tests__|testdata)(/|$)", path.lower().replace("\\", "/")):
        return "Test"
    if re.match(r"(test|_test\.)", name):
        return "Test"

    CODE_EXT = {
        ".py", ".java", ".c", ".cc", ".cpp", ".h", ".hpp", ".cs", ".go", ".rs", ".rb", ".kt", ".kts",
        ".swift", ".m", ".mm", ".scala", ".php", ".js", ".jsx", ".ts", ".tsx", ".sh", ".bash", ".zsh",
        ".r", ".m4", ".pl", ".lua", ".sql", ".css", ".html", ".xml", ".yaml", ".yml", ".gradle", ".kt",
    }
    if pname.suffix.lower() in CODE_EXT:
        return "Source"
    return "Other"
```

Classifying files into categories.

```

def mine_repo(owner_repo: str, out_rows: List[Dict], repos_base: Path, max_commits: Optional[int]) -> None:
    local_path = clone_repo(owner_repo, repos_base)
    counted = 0
    for commit in Repository(str(local_path), only_no_merge=True).traverse_commits():
        parent_shas = commit.parents
        if not parent_shas:
            continue
        parent = parent_shas[0]
        for m in commit.modified_files:
            old_path = m.old_path
            new_path = m.new_path
            pathspec = safe_pathspec(old_path, new_path)
            if not pathspec:
                continue
            patch_myers = run_git_diff(local_path, parent, commit.hash, pathspec, "myers")
            patch_hist = run_git_diff(local_path, parent, commit.hash, pathspec, "histogram")
            nm = normalize_patch(patch_myers)
            nh = normalize_patch(patch_hist)
            discrepancy = "Yes" if nm != nh else "No"
            category = classify_category(new_path or old_path)
            out_rows.append({
                "repo": owner_repo,
                "old_file_path": old_path or "",
                "new_file_path": new_path or "",
                "commit_sha": commit.hash,
                "parent_commit_sha": parent,
                #"commit_message": commit.msg.strip().replace("\n", " "),
                "commit_message": commit.msg.encode("utf-8", errors="replace").decode("utf-8"),
                "diff_myers": patch_myers,
                "diff_hist": patch_hist,
                "category": category,
                "Discrepancy": discrepancy,
            })
        counted += 1
    if max_commits and counted >= max_commits:
        break

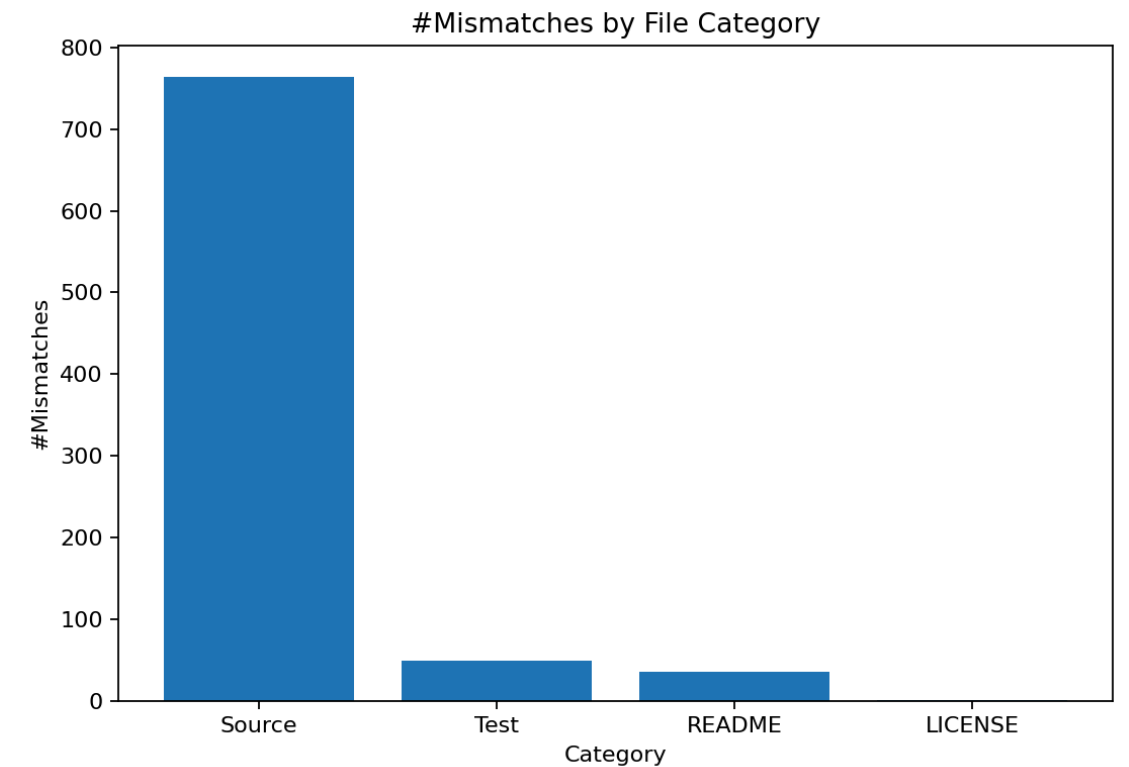
```

The main mining function, for each commit in a repository, gets the parent commit. For each of the modified files, it runs get diff with Myers and Histogram, normalizes both the patches, compares them for discrepancies, and categorizes the files.

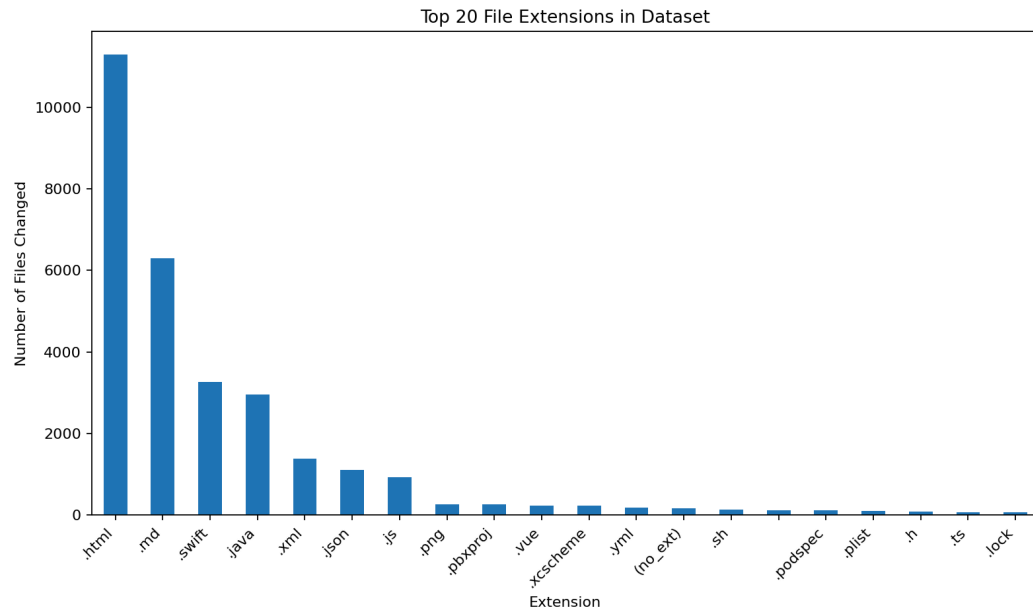
Then the final dataset is saved with another CSV file, which contains the mismatch data and a bar plot for the same.

RESULTS AND ANALYSIS

The following are the results obtained from the csv (figures and data)

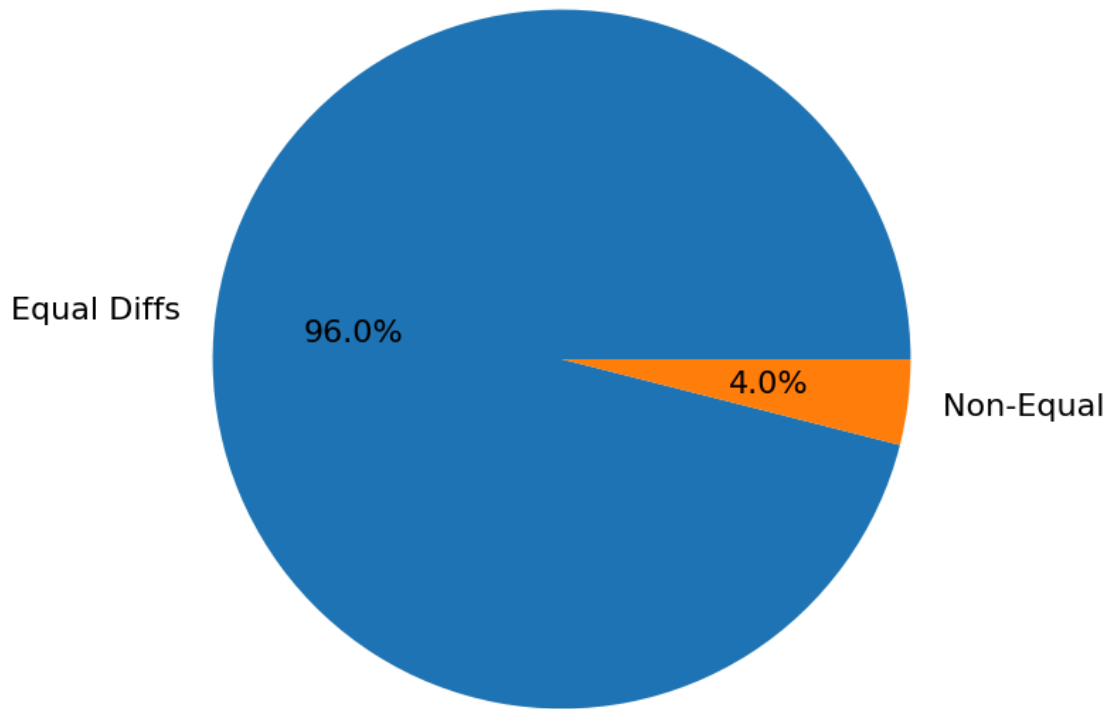


category	num_mismatches
Source	764
Test	49
README	35
LICENSE	1

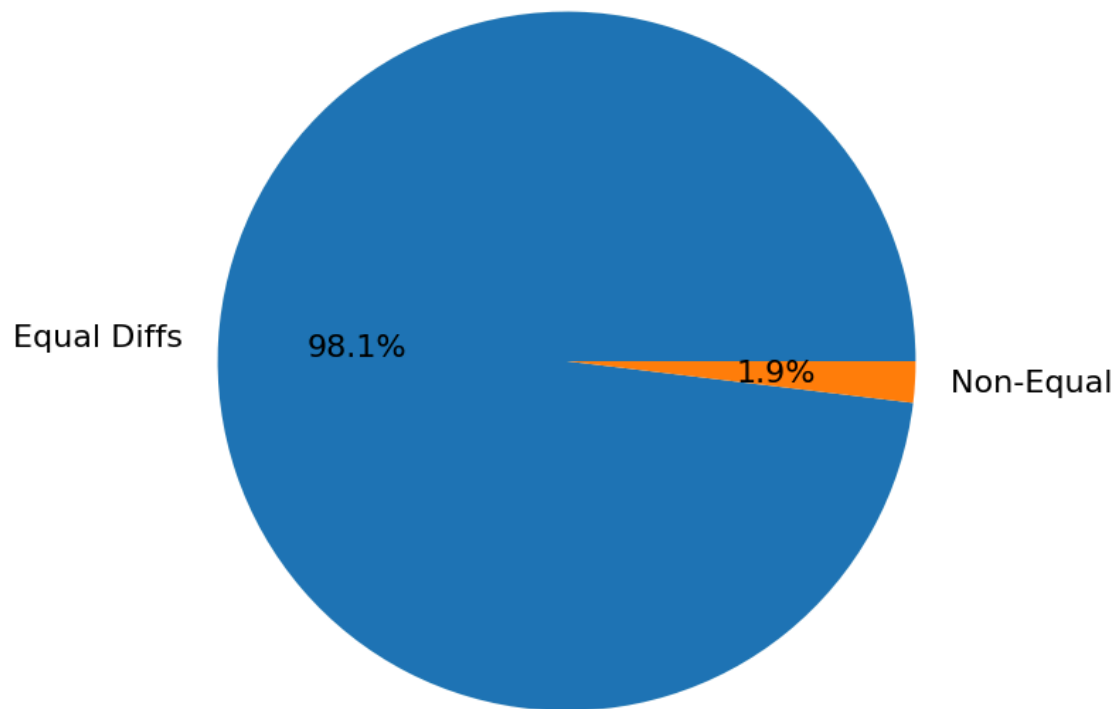


file_extension	count
.html	11295
.md	6302
.swift	3264
.java	2949
.xml	1385
.json	1108
.js	937
.png	262
.pbxproj	260
.vue	230
.xcscheme	225
.yaml	186
(no_ext)	160
.sh	137
.podspec	117
.plist	100
.h	80
.ts	70
.lock	69

Diffs for Code Artifacts

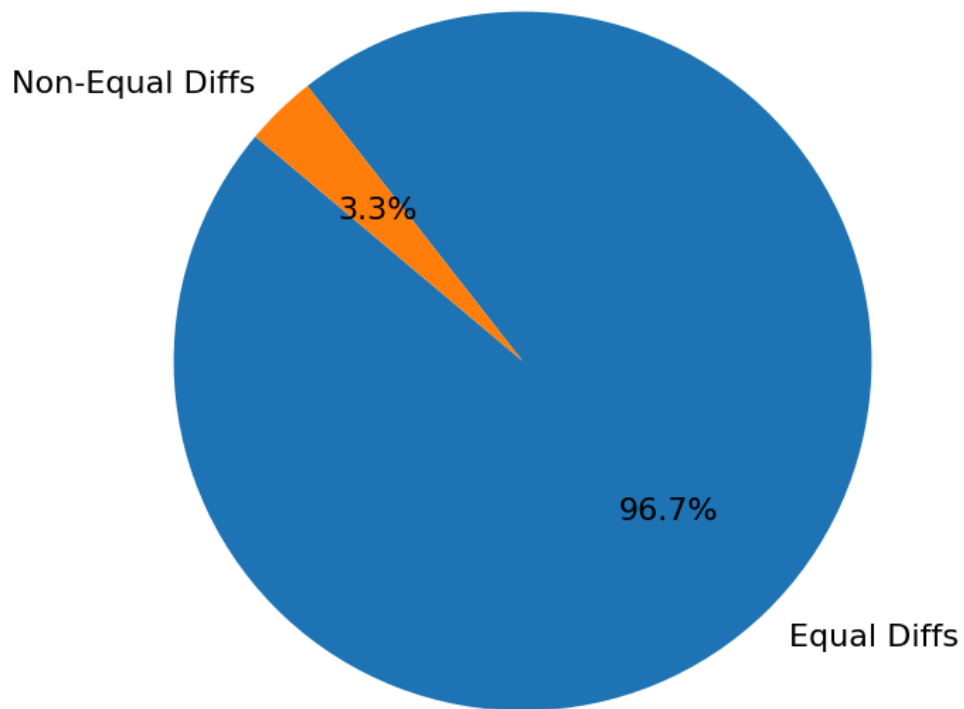


Diffs for Non-Code Artifacts



The above figure shows the obtained values for matches and non-matches for code and non-code artifacts. For coding artifact the majority both of the algorithms gives the same diff, however there is some inequalities between them (4% and 1.8% for non-code artifacts).

Equal vs Non-Equal Diffs (All Files)



Overall Equal vs Non-Equal Diffs, This shows that both myers and histogram algorithm, more or less gives the same diff, thus both of the algorithms give nearly the same results and are nearly indifferent from each other.

CONCLUSION

In this lab, I learned how to use the PyDriller API to mine GitHub repositories and study their commit history. I also compared the differences found by the Myers and Histogram diff algorithms. From this, I noticed that in most cases the results of both algorithms were very similar, with only a few differences.

REFERENCES:

[1]  CS202-LAB-4-25_08_2025.pdf

[2] [Pydriller Documentation](#)

LINKS

GitHub: [CS202_A1](#)

Drive (CSV files): [CS202_A1](#)